Lecture 03.10: Loops Using For (The Iterator Protocol)

📌 Executive Summary

Definite iteration in Python, iterable vs iterator distinction, CPython FOR_ITER bytecode, and sequence traversal.

🔬 Theoretical Foundations & Principles

1. Definite Iteration via Iterator Protocol

Unlike traditional C-style index loops (`for (int i=0; i<N; i++)`), Python's `for` loop is a **collection traversal loop** governed by the Iterator Protocol:

```
for item in iterable:
    consume(item)
```

Equivalent CPython execution under the hood:

```
_iter = iter(iterable)
while True:
    try:
        item = next(_iter)
    except StopIteration:
        break
    consume(item)
```

2. Iterator Protocol Axioms

An iterable object implements `__iter__()` returning an iterator. An iterator implements `__next__()` returning successive items or raising `StopIteration`.

💻 Production Implementation & Code

```
# Demonstrating underlying iterator protocol
fruits = ["apple", "banana", "cherry"]
iterator = iter(fruits)
```

```
print("First item:", next(iterator))
print("Second item:", next(iterator))
print("Third item:", next(iterator))
# next(iterator) now would raise StopIteration
```

⚙️ Algorithmic Complexity & Systems Analysis

- **Time Complexity:** Detailed in respective algorithmic operations.
- **Auxiliary Space:** Memory bounds and CPython frame management.